

Extended Abstract

Motivation GRPO (Group Relative Policy Optimization) DeepSeek-AI et al. (2025) is a recently proposed RL method used in models like DeepSeekMath to fine-tune LLMs for reasoning. These approaches reward correct completions but do not explicitly regulate exploration (e.g., entropy of sampled responses). Recent work suggests that token-level confidence can be used as a dense intrinsic signal. We aim to understand how confidence interacts with correctness and whether we can steer model behavior more precisely.

Method Current policy-based fine-tuning methods for reasoning do not control how much models exploit/explore: they reward correct answers but do not explicitly encourage the model to commit or search the solution space. We attempt to reward models based on confidence level via an information-theoretic extension of Group Relative Policy Optimisation (GRPO) that adds a *confidence bonus*; if the model is more confident (low entropy), then it is rewarded. This reward does not need any external reward whatsoever, and operates just on the models own outputs. The hypothesis here is that GRPO samples multiple completions, which acts as an exploratory phase- we then reward the model for being confident about its answer. Results on GSM8K suggest that promoting confident responses does help the model learn to improve its performance.

Implementation We fine-tune a 1B parameter GEMMA-3 INSTRUCT model with GRPO DeepSeek-AI et al. (2025). Let $(x_i, y_i)_{i=1}^B$ denote a batch of prompts and reference answers. For each prompt we sample G completions $c_{i,g}$. For comparison, we train a model using just GRPO with no modifications; we compute reward $r_{i,g}$ via *exact format*, *approximate format*, and *answer correctness* checks, and form relative advantages $a_{i,g}$. We also have another run where we modify the original objective by adding a confidence bonus $\alpha \cdot H(\pi(c | x))$ that penalizes uncertain final predictions. Runs are done on a single A100, with 4 generations per prompt, and using GSM8K as the dataset.

Results We obtain a negative result- the confidence based approach leads to a slight degradation of GSM8K performance from 0.25 to 0.15. However, qualitatively, we observe that this is mostly due to incorrect formatting. The reward does increase through training, suggesting that the model is able to learn from the training signal.

Discussion And Conclusion The results suggest that a good course of action for future work is to try confidence-based rewards, except with formatting rewards as the only external rewards. Alternatively, one can try to modify the method of extracting the answer from the response (right now, it needs to be wrapped in specific tags). This work shows the potential of information-theoretic methods (without external rewards), and could lead to promising ways of incorporating notions of confidence and exploration/exploitation tradeoffs in RL-based model fine-tuning.

Exploration and Confidence in Large Reasoning Models

Nikil Ravi

Department of Computer Science
Stanford University
nravi@stanford.edu

Abstract

Recently, researchers have been exploring the use of reinforcement learning (RL) algorithms to post-train and fine-tune large language models (LLMs). For example, OpenAI’s o1 series and Deepseek R1 apply RL to improve LLM performance on math problem-solving. However, strategies such as GRPO DeepSeek-AI et al. (2025) do not explicitly control the level of exploration or exploitation. Questions about how these RL techniques applied to LLMs explore/exploit are understudied. In this project, we study the impact of rewarding confidence in model responses when sampling completions in GRPO- the intuition here is that GRPO samples several times, acting as a way to explore, and we now reward the model’s ability to be confident in each of its responses. Results on GSMK8 show that fine-tuning the model by rewarding confidence (with no external rewards) does improve performance over the base model, and achieves results comparable with a model trained via GRPO in the usual way.

1 Introduction

2 Related Work

The training of large language models has typically involved RL (reinforcement learning), but only for post-training or instruction following Ouyang et al. (2022) Rafailov et al. (2024). More recently, large language models have recently been incentivized to perform chain-of-thought reasoning, with models such as OpenAI’s o1 (and o-series generally), DeepSeek-R1 DeepSeek-AI et al. (2025) and many other models utilizing reinforcement learning to achieve this outcome. However, while methods such as GRPO have been proposed and shown to work, there is not a good understanding of (a) why they work well and (b) how we can control exploration and exploitation to achieve better outcomes. Recent work such as Muennighoff et al. (2025) has shown that a small amount SFT and/or well-chosen prompting strategies can encourage exploration and ultimately boost reasoning ability. Some preliminary work has also attempted to study chain-of-thought reasoning and LLMs generally via the lens of information theory Skean et al. (2025).

3 Method

As a baseline approach, we use the GRPO algorithm DeepSeek-AI et al. (2025). We use the sum of the following rewards:

- *match_format_exactly*: Rewards completions that strictly follow the expected output format

- *match_format_approximately*: Gives partial credit for loosely matching the expected format, even if not perfectly aligned.
- *check_answer*: Binary reward (1 or 0) based on exact match between the predicted answer and the ground-truth answer.
- *check_numbers*: Rewards numeric agreement between predicted and target answers, even if formatting differs (e.g., ignoring units or whitespace).

Rewards are of the order of +1, +0., -0.5, etc.; we refer readers to the code to check the exact reward magnitudes for each case.

With our proposed method, named *GRPO_conf*, we modify this reward. To encourage confident predictions without requiring ground-truth answers, we define a reward based on the entropy of the model’s token distributions. For each generated completion, we run a forward pass through the model and compute the entropy over the predicted vocabulary distribution at each token. We then average the entropy over the final k tokens of the completion. The reward is the negative average entropy, which favors sharper and more "confident" token distributions. This method provides unsupervised (i.e., no external rewards needed) training signal and aligns with recent approaches like RENT. We log correctness (based on string match with reference answers) alongside the entropy reward to analyze how confidence correlates with accuracy.

Algorithm 1 Compute Entropy-Based Confidence Reward

Require: Prompts $P = \{p_i\}$, completions $C = \{c_i\}$, tokenizer $\mathcal{T}$, model $\mathcal{M}$, last k tokens, (optional) answers $A = \{a_i\}$

Ensure: Rewards $R = \{r_i\}$

```

1: Initialize  $R \leftarrow []$ 
2: for  $i = 1$  to  $|P|$  do
3:    $x_i \leftarrow \text{flatten}(p_i) + \text{flatten}(c_i)$ 
4:    $\text{input\_ids} \leftarrow \mathcal{T}(x_i)$ 
5:    $\text{logits} \leftarrow \mathcal{M}(\text{input\_ids})$ 
6:    $\text{probs} \leftarrow \text{softmax}(\text{logits})$ 
7:    $\text{log\_probs} \leftarrow \text{log\_softmax}(\text{logits})$ 
8:    $H \leftarrow -\sum_v \text{probs} \cdot \text{log\_probs}$  // token-level entropy
9:   Determine completion start index  $s$ 
10:   $H_{\text{comp}} \leftarrow H[s:]$ 
11:   $\text{avg\_H} \leftarrow \text{mean}(H_{\text{comp}}[-k:])$ 
12:   $r_i \leftarrow -\text{avg\_H}$ 
13:  if  $a_i$  is provided then
14:     $\text{correct} \leftarrow \mathbb{I}\{a_i \in c_i\}$ 
15:    Log  $(r_i, \text{avg\_H}, \text{correct})$  to WandB
16:  else
17:    Log  $(r_i, \text{avg\_H})$  to WandB
18:  end if
19:  Append  $r_i$  to  $R$ 
20: end for
21: return  $R$ 

```

4 Experimental Setup

I used the GSM8K dataset Cobbe et al. (2021) for training/fine-tuning a Gemma Instruct 1B parameter model (link). I used <https://unsloth.ai> for fine-tuning infrastructure management and efficient inference + finetuning. Experiments were run on a single A100. I used LoRA (low-rank adaptation) Hu et al. (2021) for parameter efficient fine-tuning, in order to optimize training time and efficiency; I used vLLM for efficient inference.

Hyperparameters used for fine-tuning are also provided in the table below.

For standard GRPO, I used multiple functions to assign rewards: *match_format_exactly*, *match_format_approximately*, *check_answer*, and *check_numbers*. These rewards use formatting and exact checks to compute rewards.

Hyperparameter	Value
Learning Rate	5e-6
Adam Beta1	0.9
Adam Beta2	0.99
Weight Decay	0.1
Warmup Ratio	0.1
LR Scheduler	cosine
Optimizer	adamw_torch_fused
Per-device Train Batch Size	1
Number of Generations	4
Max Prompt Length	256
Max Completion Length	768

Table 1: GRPO Training Hyperparameters

5 Results

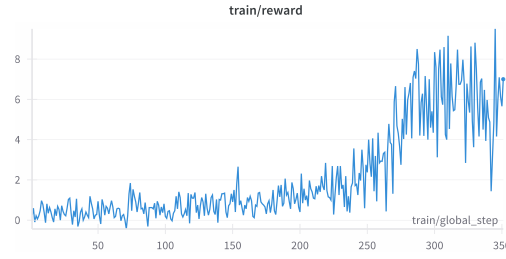


Figure 1: Reward using the standard GRPO method for fine-tuning

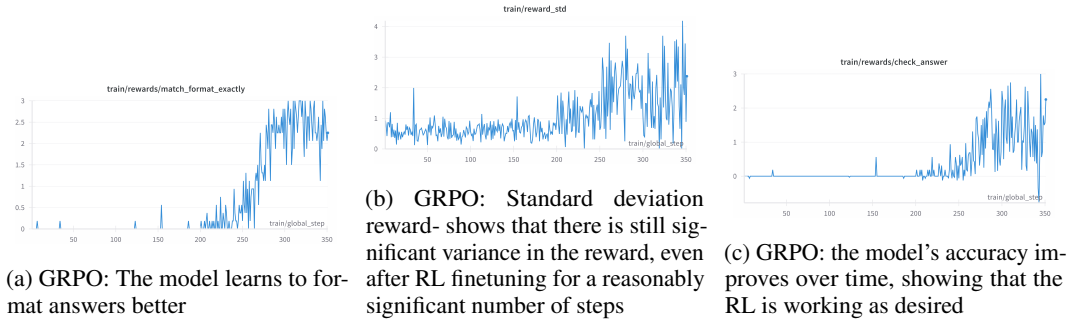


Figure 2: Evaluation results of the fine-tuned model on the GSM8K test set.



Figure 3: Evaluation results of the fine-tuned model on the GSM8K test set.

5.1 Quantitative Evaluation

Table 2: Performance Comparison

Method	GSM8K accuracy
Gemma 3 1B	0.25
Gemma 3- GRPO	0.51
Gemma 3- GRPO-conf	0.15

5.2 Qualitative Analysis and Discussion

We note that our method results in the model scoring lower on GSM8K on the test set; however, qualitatively, the responses are still reasonable in terms of correctness; a lot of the accuracy points lost are due to incorrect formatting. Thus, a good future direction is to combine formatting rewards with confidence-based rewards.

6 Team Contributions

- **Nikil Ravi:** Trained models using GRPO and GRPO with confidence rewards, ran tests

Changes from Proposal Initially, I had proposed adding a term to encourage exploration. While still related, my work here reinforces confidence, and it quite different from my initial proposal. However, by weighting this term or multiplying this term by -1 to encourage doubtful responses, it could be argued that we’d be incentivizing the model to learn to output responses where it is doubtful, leading to more exploration. Thus, the same method can *also be applied to encourage exploration!*

References

- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168 [cs.LG] <https://arxiv.org/abs/2110.14168>
- DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojuan Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Shengfeng Ye, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wanbiao Zhao, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanhong Xu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha,

- Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. arXiv:2501.12948 [cs.CL] <https://arxiv.org/abs/2501.12948>
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685 [cs.CL] <https://arxiv.org/abs/2106.09685>
- Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. arXiv:2501.19393 [cs.CL] <https://arxiv.org/abs/2501.19393>
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. arXiv:2203.02155 [cs.CL] <https://arxiv.org/abs/2203.02155>
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2024. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. arXiv:2305.18290 [cs.LG] <https://arxiv.org/abs/2305.18290>
- Oscar Skea, Md Rifat Arefin, Dan Zhao, Niket Patel, Jalal Naghiyev, Yann LeCun, and Ravid Shwartz-Ziv. 2025. Layer by Layer: Uncovering Hidden Representations in Language Models. arXiv:2502.02013 [cs.LG] <https://arxiv.org/abs/2502.02013>